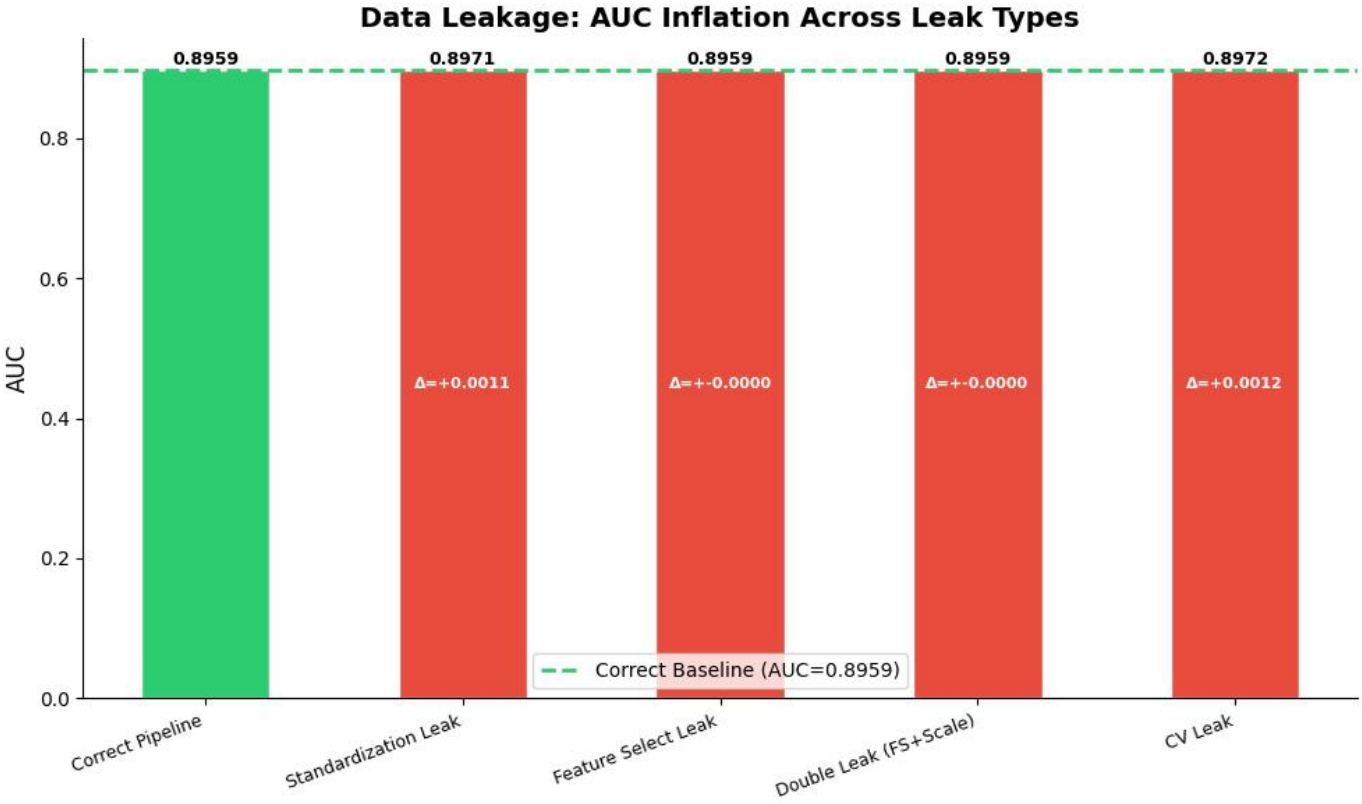


07_data_leakage 数据泄漏分析

泄漏类型对比汇总:

Correct Pipeline	AUC=0.8959	($\Delta=+0.0000$)
Standardization Leak	AUC=0.8971	($\Delta=+0.0011$)
Feature Select Leak	AUC=0.8959	($\Delta=-0.0000$)
Double Leak (FS+Scale)	AUC=0.8959	($\Delta=-0.0000$)
CV Leak	AUC=0.8972	($\Delta=+0.0012$)



实验 1 & 2: 标准化泄漏

错误流程

```
# ❌ 错误：在全数据上标准化
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_full) # 用了测试集的均值/标准差！
X_train, X_test = train_test_split(X_scaled, ...)
```

泄漏了什么？

信息	说明
μ (均值)	测试集 Age 的均值被泄露给了训练集
σ (标准差)	测试集 Age 的标准差被泄露给了训练集

你的模型:

Python

```
LogisticRegression()
```



运行

本质是在学习:

$$P(y = 1) = \frac{1}{1 + e^{-(w_1x_1 + w_2x_2 + \dots + b)}}$$

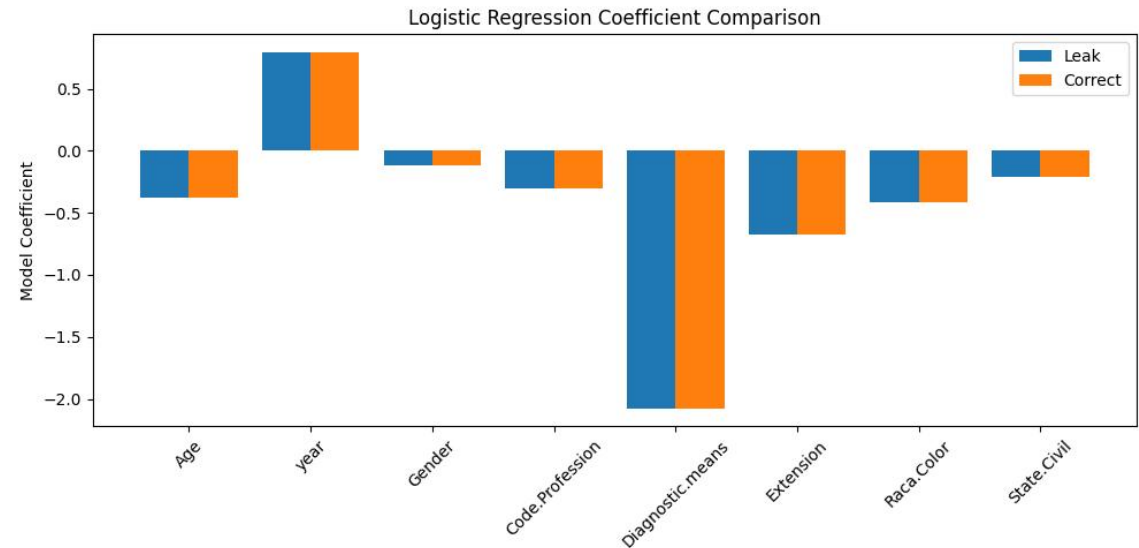
其中:

- x : 特征
- w : 模型系数 (coef)
- b : 截距

Coef 是通过最大似然估计求解损失函数得到的, 其大小不仅取决于标签 y , 还取决于输入特征矩阵 X 。标准化泄露改变了均值和标准差, 使输入数据 X 的数值空间发生变化, 从而改变了模型优化的目标函数。

Coef 表示该特征对预测结果的贡献大小和方向 (正/负)

	Feature	Leak_coef	Correct_coef	Difference	Relative_change(%)
0	Age	-0.378099	-0.377517	-0.000582	0.154126
1	year	0.796697	0.795694	0.001003	0.126004
2	Gender	-0.117391	-0.117353	-0.000038	0.032670
3	Code.Profession	-0.304182	-0.303084	-0.001098	0.362175
4	Diagnostic.means	-2.078558	-2.074187	-0.004371	0.210726
5	Extension	-0.676395	-0.675012	-0.001383	0.204813
6	Raca.Color	-0.410839	-0.411898	0.001058	0.256970
7	State.Civil	-0.208839	-0.208883	0.000044	0.021258



实验 3: 特征选择泄漏

▶ 不同 k 值下的泄漏效应:

k=2: 泄漏	AUC=0.8543		正确	AUC=0.8543		虚高=0.0000
k=3: 泄漏	AUC=0.8830		正确	AUC=0.8830		虚高=0.0000
k=4: 泄漏	AUC=0.8888		正确	AUC=0.8888		虚高=0.0000
k=5: 泄漏	AUC=0.8952		正确	AUC=0.8952		虚高=0.0000
k=6: 泄漏	AUC=0.8990		正确	AUC=0.8989		虚高=0.0000
k=7: 泄漏	AUC=0.8990		正确	AUC=0.8990		虚高=0.0000

改变 SelectKBest(k) 参数

研究“特征选择规模是否影响泄露”

增加交叉验证次数 (08)

研究“评价稳定性是否掩盖泄露”

CV_Folds	Correct_Mean_AUC	Correct_STD	Leak_Mean_AUC	Leak_STD	Delta_AUC
3	0.885213	0.001454	0.885219	0.001460	0.000007
5	0.885237	0.002766	0.885247	0.002759	0.000010
10	0.885156	0.004715	0.885169	0.004707	0.000013
20	0.885191	0.007819	0.885190	0.007824	-0.000002

误区：认为“增加交叉验证次数会导致更多数据参与模型训练，因此可能加剧数据泄露并进一步提高AUC”。

这种理解混淆了**模型评价稳定性**与**数据泄露问题**。

交叉验证的作用是通过多次划分训练集和验证集，降低单次数据划分造成的随机波动，**提高模型性能估计的可靠性**。

交叉验证次数的增加**并不会改变已经获得的信息，也不会增强或消除特征泄露影响**。本实验通过比较不同交叉验证策略下的结果发现，3-fold、5-fold、10-fold和20-fold条件下，正确流程与特征选择泄露流程的AUC差异均接近于0，说明在当前低维、大样本癌症数据中，特征选择结果具有较高稳定性，**泄露虽然存在，但未造成明显AUC虚高**。

正确理解应为：**CV次数影响的是性能估计的稳定程度，而泄露程度取决于信息是否发生越界，两者属于不同层面的问题**。

因为F统计量依赖：

样本量 n

ANOVA公式：

$$F = \frac{MS_{between}}{MS_{within}}$$

其中：

$$MS_{between} = \frac{SS_{between}}{k - 1}$$

$$MS_{within} = \frac{SS_{within}}{N - k}$$

当样本增加：

- 均值估计更准确
- 方差估计更稳定
- 随机误差降低

F： 衡量某个特征与目标变量关联强弱的统计量

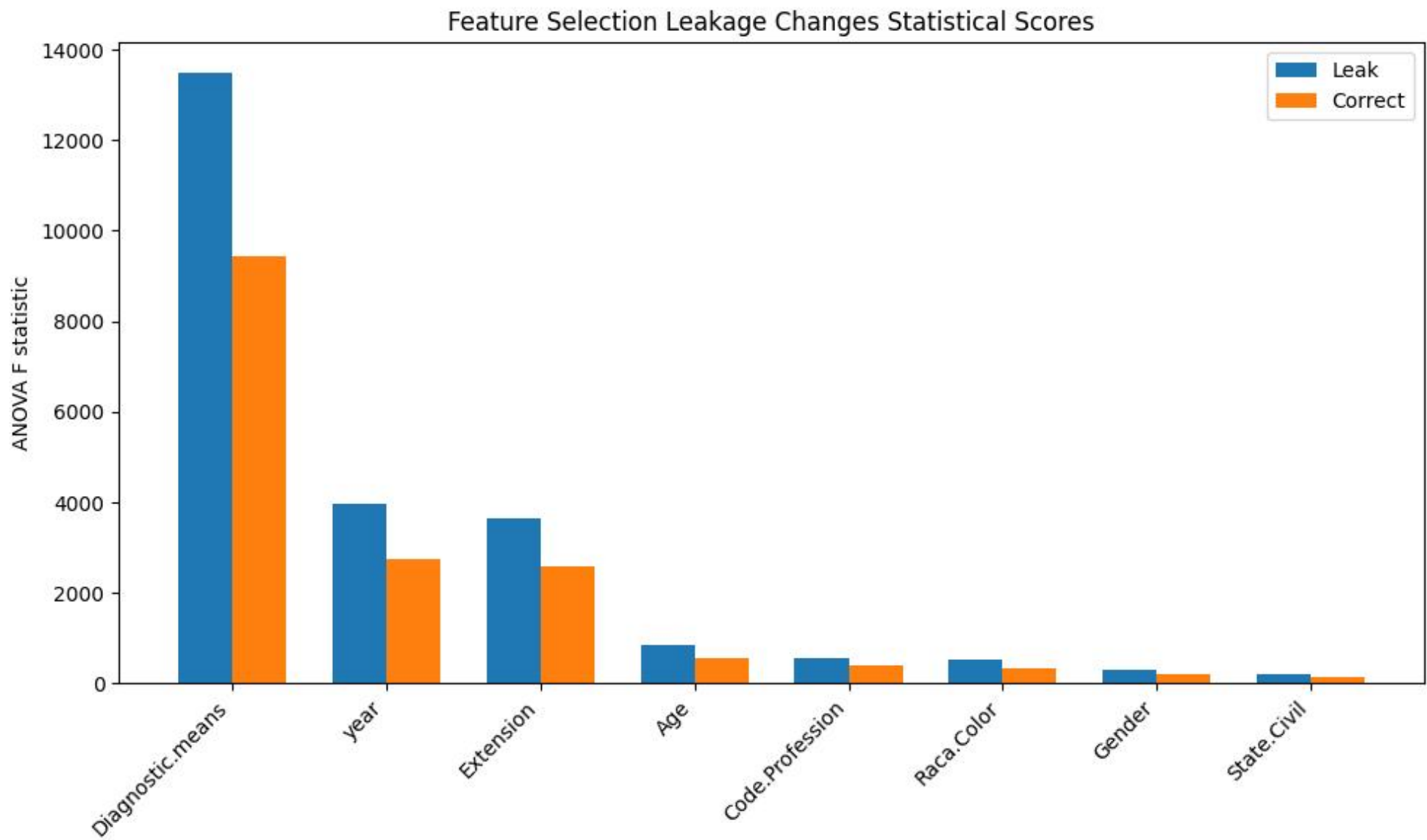
F越大，说明不同类别在这个特征上的差异越明显，而随机波动越小，因此这个特征越有区分能力。

F统计量比较:

	Feature	Leak_F	Correct_F	Leak_P	Correct_P	Increase(%)
4	Diagnostic.means	13476.342619	9445.860249	0.000000e+00	0.000000e+00	42.669299
1	year	3962.890701	2736.907446	0.000000e+00	0.000000e+00	44.794473
5	Extension	3658.885168	2573.985057	0.000000e+00	0.000000e+00	42.148656
0	Age	858.934099	570.020016	3.513662e-186	2.517917e-124	50.684901
3	Code.Profession	567.156194	413.448852	3.324636e-124	4.898648e-91	37.176870
6	Raca.Color	532.794156	346.066182	7.282971e-117	1.258548e-76	53.957302
2	Gender	290.353582	206.998832	8.409663e-65	1.034886e-46	40.268222
7	State.Civil	204.191521	148.645237	3.604864e-46	4.470188e-34	37.368358

► 不同 K 值下的泄漏效应:

k=2: 泄漏	AUC=0.8543		正确	AUC=0.8543		虚高	=0.0000
k=3: 泄漏	AUC=0.8830		正确	AUC=0.8830		虚高	=0.0000
k=4: 泄漏	AUC=0.8888		正确	AUC=0.8888		虚高	=0.0000
k=5: 泄漏	AUC=0.8952		正确	AUC=0.8952		虚高	=0.0000
k=6: 泄漏	AUC=0.8990		正确	AUC=0.8989		虚高	=0.0000
k=7: 泄漏	AUC=0.8990		正确	AUC=0.8990		虚高	=0.0000



实验1：标准化泄漏

不要看F。

看：

指标	意义	📄
μ 变化	泄漏证据	
σ 变化	泄漏证据	
标准化后数值差异	影响程度	
模型coef变化	模型影响	
预测概率差异	最终影响	

结论：

标准化泄漏确实改变了模型输入，但由于只泄漏均值和标准差两个统计量，影响有限，因此AUC几乎不变。

实验3：特征选择泄漏

看：

指标	意义
F值变化	统计推断变化
P值变化	显著性变化
特征排名	是否改变选择
AUC	最终效果

结论：

特征选择泄漏改变了统计量，但由于低维数据中特征排名稳定，因此AUC仍未变化。

5.3 防止泄漏的最佳实践

方法 1: 使用 Pipeline (推荐)

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest
from sklearn.linear_model import LogisticRegression

# Pipeline 确保每折各自独立做预处理
pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('select', SelectKBest(k=6)),
    ('lr', LogisticRegression(class_weight='balanced'))
])

# 交叉验证自动为每折创建独立的预处理流程
scores = cross_val_score(pipe, X_train, y_train, cv=5)
```

方法 2: 数据划分后立即锁住测试集

```
# 实战技巧: 将测试集“锁”在另一个文件/变量中
X_train, X_test, y_train, y_test = train_test_split(...)

# 在完成所有预处理、特征工程、特征选择之前,
# 绝对不再碰 X_test / y_test
# 可以把测试集写到一个加密文件中, 只有在最终评估时才读取
```